

Dance4Life: Implementação de um Sistema Multiagente para a Promoção do Envelhecimento Ativo

Carlos da Mota Bergueira (pg11605@alunos.uminho.pt)
Diego Jefferson Mendes Silva (pg59999@alunos.uminho.pt)
Filipa Araújo Pereira (pg42201@alunos.uminho.pt)
Rui Manuel Martins Marques Rodrigues (pg7942@alunos.uminho.pt)

Universidade do Minho, Braga, Portugal

Abstract. O presente relatório descreve a conceção, modelação e implementação do Dance4Life, um Sistema Multiagente (SMA) distribuído, desenvolvido em Python com a *framework* SPADE (Smart Python Agent Development Environment), aplicado ao domínio do envelhecimento ativo. O sistema tem como objetivo suportar mecanismos de monitorização de atividade física, enriquecimento contextual e emparelhamento social através de uma arquitetura modular baseada em agentes especializados. A solução integra uma API REST (Application Programming Interface - Representational State Transfer) com um ecossistema multiagente responsável pela recolha, coordenação, processamento e persistência distribuída de dados. A comunicação entre agentes é realizada sobre XMPP (Extensible Messaging and Presence Protocol), seguindo princípios compatíveis com FIPA-ACL (Foundation for Intelligent Physical Agents - Agent Communication Language), através da utilização de performativas, ontologias e identificadores de conversação. A troca de informação é suportada por *payloads* estruturados serializados com jsonpickle. Os resultados obtidos demonstram a viabilidade da abordagem multiagente para a orquestração assíncrona de *pipelines* distribuídos em cenários de envelhecimento ativo. O relatório apresenta ainda uma análise crítica das limitações identificadas na implementação atual, nomeadamente ao nível da conformidade protocolar, validação semântica e acoplamento estático entre agentes, propondo futuras melhorias orientadas à escalabilidade, robustez e coordenação distribuída.

Keywords: Sistemas Multiagente · SPADE · FIPA-ACL · Envelhecimento Ativo · Dance4Life · Human Activity Recognition (HAR)

1 Introdução

O presente trabalho insere-se no domínio dos Sistemas Multiagente aplicados ao envelhecimento ativo, explorando a utilização de arquiteturas distribuídas para monitorização, coordenação e processamento de dados em ambientes inteligentes. Esta secção apresenta o contexto e motivação do problema, os objetivos definidos para o projeto e a estrutura geral do relatório.

1.1 Domínio e Motivação

O envelhecimento demográfico global constitui um dos principais desafios sociais e de saúde pública da atualidade. A estimativa de que, até 2050, a população com mais de 65 anos atinja 1,5 mil milhões de pessoas impõe uma mudança de paradigma, promovendo a transição de modelos centrados em cuidados clínicos hospitalares para abordagens de envelhecimento ativo e monitorização remota no contexto residencial. É neste enquadramento que surge o ecossistema concetual Dance4Life, proposto como domínio de aplicação deste trabalho prático. O Dance4Life pretende promover o envelhecimento ativo através da inferência implícita de preferências dos utilizadores, correlacionando padrões de dança e interações sociais. A diversidade e heterogeneidade dos dados envolvidos, incluindo sensores inerciais, sinais vitais e perfis sociais, dificultam a adoção de arquiteturas monolíticas tradicionais. Neste contexto, os Sistemas Multiagente emergem como um paradigma tecnológico adequado, devido à sua natureza modular, distribuída e assíncrona. Esta abordagem permite que entidades computacionais autónomas, designadas agentes, cooperem entre si para perceber o ambiente, interpretar dados heterogêneos e suportar mecanismos de decisão em tempo real.

1.2 Objetivos

O principal objetivo deste trabalho consiste na conceção e implementação de um protótipo distribuído baseado em Sistemas Multiagente, aplicado ao domínio do envelhecimento ativo e desenvolvido em Python com recurso à *framework* SPADE. A solução proposta procura explorar as capacidades de coordenação distribuída, comunicação assíncrona e modularidade associadas ao paradigma multiagente. Neste contexto, pretende-se desenvolver uma arquitetura distribuída suportada por metodologias de modelação Agent UML (AUML), assegurando uma organização modular dos diferentes componentes do sistema. O projeto contempla igualmente a implementação de vários agentes especializados, nomeadamente SensorAgent, CoordinatorAgent, HARAgent, EnvironmentAgent, DatabaseAgent e ClusteringAgent, recorrendo a behaviours do tipo CyclicBehaviour, OneShotBehaviour e PeriodicBehaviour. Paralelamente, procura-se garantir a utilização de mecanismos de comunicação compatíveis com os princípios FIPA-ACL, incluindo performativas, ontologias e identificadores de conversação. Outro dos objetivos passa pela utilização de *payloads* estruturados serializados com jsonpickle, promovendo maior consistência semântica na troca de informação entre agentes. Por fim, pretende-se implementar mecanismos de coordenação assíncrona capazes de suportar o processamento distribuído do *pipeline* de dados sem bloqueio da thread principal da aplicação.

1.3 Estrutura do Relatório

O relatório encontra-se organizado da seguinte forma: a Secção 2 apresenta a conceção e modelação do sistema, incluindo a arquitetura, os agentes e os protocolos de comunicação; a Secção 3 descreve a implementação técnica e o funcionamento

do *pipeline* de dados; a Secção 4 apresenta os resultados obtidos e a respetiva análise crítica; por fim, a Secção 5 reúne as conclusões do trabalho e as principais recomendações para desenvolvimento futuro.

1.4 Fundamentação Teórica

O enquadramento teórico deste trabalho baseia-se nos materiais pedagógicos da unidade curricular [19–22], com destaque para o texto de Novais e Analide [22] sobre Agentes Inteligentes e Sistemas Multiagente, complementado por referências clássicas da área. Segundo Wooldridge [23], um agente corresponde a um sistema computacional capaz de atuar de forma autónoma e flexível num determinado ambiente. A noção fraca de agente caracteriza-se por quatro propriedades principais [24]: autonomia, reatividade, pró-atividade e sociabilidade. No contexto do Dance4Life, estas propriedades refletem-se diretamente na arquitetura implementada. Os agentes executam tarefas de forma independente, reagem a mensagens ACL recebidas através dos behaviours do SPADE e cooperam entre si ao longo do *pipeline* distribuído de processamento. Relativamente às arquiteturas de agentes, Novais e Analide [22] identificam três paradigmas principais: deliberativo, reativo e híbrido. O paradigma deliberativo assenta em mecanismos de representação e planeamento interno; o paradigma reativo baseia-se em respostas diretas a estímulos externos; e o paradigma híbrido combina ambas as abordagens. Esta classificação será posteriormente aplicada aos diferentes agentes do sistema na Secção 2.2. Um Sistema Multiagente corresponde a um conjunto de entidades autónomas que cooperam para resolver problemas que ultrapassam as capacidades individuais de cada agente [25, 26]. A comunicação entre agentes requer uma plataforma de comunicação, uma linguagem de interação e uma ontologia partilhada [22]. No Dance4Life, estes requisitos são assegurados através do protocolo XMPP, da semântica FIPA-ACL e das ontologias `sensor_activity`, `movement_recommendation` e `matching`. A coordenação adotada é de natureza cooperativa, uma vez que todos os agentes partilham o objetivo comum de processar e persistir dados relacionados com a atividade do utilizador.

2 Conceção e Modelação do Sistema

A fase de conceção do sistema seguiu os princípios da metodologia de modelação baseada em agentes (AUML) [22]. O desenvolvimento de um ecossistema orientado ao envelhecimento ativo exige o processamento distribuído de dados heterogêneos em tempo real, incluindo informação fisiológica, inercial e contextual. Neste contexto, a arquitetura foi concebida segundo o paradigma dos Sistemas Multiagente descrito por Weiss [25], privilegiando a distribuição funcional, a modularidade e a cooperação entre entidades autónomas. A decomposição das responsabilidades por agentes especializados permite reduzir o acoplamento entre componentes e facilitar a escalabilidade e evolução do sistema, em contraste com abordagens monolíticas centralizadas.

2.1 Arquitetura do Sistema Multiagente (Topologia)

A arquitetura do Dance4Life foi concebida segundo o paradigma distribuído *edge-cloud*, procurando equilibrar três requisitos fundamentais e frequentemente conflitantes em sistemas de saúde digital: a latência de resposta às intervenções, a preservação da privacidade dos dados biométricos e a escalabilidade do processamento analítico. A separação de responsabilidades entre dispositivo e servidor não é, por isso, meramente técnica, reflete uma decisão de desenho fundamentada nos princípios de *Privacy by Design* exigidos pelo RGPD, ao garantir que os sinais fisiológicos brutos são processados localmente e apenas métricas derivadas atravessam a rede. A arquitetura resultante, ilustrada na Figura 1, organiza-se em cinco camadas funcionais com interfaces bem definidas. A camada de aquisição no edge é materializada em duas aplicações nativas para Android, uma para *smartphone* e outra para *smartwatch*, responsáveis pela captura contínua dos sinais inerciais (acelerómetro triaxial e giroscópio, via `TYPE_ACCELEROMETER` e `TYPE_GYROSCOPE`) e fisiológicos (frequência cardíaca via `TYPE_HEART_RATE`, com mecanismo de *fallback* baseado em estimativa por magnitudes inerciais quando o sensor não está disponível), bem como da localização geográfica enriquecida semanticamente por *reverse geocoding* (cidade, país e rua). Ambos os dispositivos partilham os mesmos contratos de abstração definidos no módulo core, as interfaces `SensorProvider`, `LocationProvider` e `DeviceProvider`, tornando o código de domínio independente da plataforma física e facilitando a extensão a outros tipos de dispositivos. O pré-processamento local é executado pelo `DanceController`, que agrega os eventos sensoriais em janelas deslizantes, computa as magnitudes euclidianas dos vetores inerciais, estima o indicador de ritmo de movimento e constrói a observação compacta `MovementObservation`, com as variáveis `activityLevel`, `physicalFatigue` e `irritationLevel`, que alimenta a política de Aprendizagem por Reforço inferida localmente via `ONNX Runtime`. A periodicidade de envio ao servidor está fixada em 30 segundos, assegurando granularidade temporal suficiente para análise longitudinal sem sobrecarregar a rede. A camada de ingestão e serviços de contexto, implementada em `Flask`, expõe uma API REST (Representational State Transfer) com cinco endpoints principais: `/collect_data_activities` para receção de telemetria de atividade; `/get_environment_data` para consulta de contexto meteorológico em tempo real junto da API `OpenWeatherMap`; `/set_movement_recommendation` para registo das recomendações geradas no edge; e `/get_user_match` e `/set_invite_status` para gestão do ciclo de vida dos convites sociais. O servidor mantém ainda um registo de agentes ativos com mecanismo de *heartbeat*, suportando a coordenação assíncrona da camada multiagente.

A orquestração multiagente, construída sobre a *framework* `SPADE` (Smart Python Agent Development Environment) seguindo as normas `FIPA-ACL` (Foundation for Intelligent Physical Agents - Agent Communication Language), implementa um *pipeline* de mensagens orientadas por ontologias e performativas, seguindo a ordem `SensorAgent` → `CoordinatorAgent` → `HARAgent` → `EnvironmentAgent` → `DatabaseAgent` para o fluxo principal de atividade. O Environ-

mentAgent enriquece o *payload* com a temperatura ambiente obtida em tempo real, enquanto o DatabaseAgent persiste o evento enriquecido no Firebase. Em paralelo, um ramo independente do CoordinatorAgent encaminha dados de atividade para o MatchingAgent, que aplica um modelo de agrupamento por níveis de atividade para gerar convites sociais entre utilizadores compatíveis.

Esta arquitetura orientada a mensagens confere elevada modularidade e permite evoluir componentes individuais sem afetar o restante *pipeline*. Por fim, a camada de persistência e analítica utiliza o Firebase como *backend* documental para armazenamento de eventos de atividade, convites, estados de matching e recomendações de movimento. Os dados persistidos são consumidos por *dashboards* analíticos implementados em PowerBI, D3.js e Leaflet, disponibilizando visualizações interativas de séries temporais, distribuição geográfica, rankings de participação e decomposição de *scores* por utilizador. Esta arquitetura em cinco camadas reflete um equilíbrio deliberado entre a capacidade de resposta em tempo real, garantida pelo processamento e inferência no edge, e a riqueza analítica proporcionada pela persistência centralizada. A adoção do paradigma multiagente com SPADE não se limita a uma conveniência de implementação; decorre da necessidade de coordenar comportamentos heterogêneos (reconhecimento de atividade, enriquecimento contextual, matching social e persistência) de forma assíncrona, auditável e com separação clara de responsabilidades. Do ponto de vista da privacidade, o facto de os sinais biométricos brutos nunca abandonarem o dispositivo, sendo apenas transmitidas métricas derivadas com granularidade de 30 em 30 segundos, concretiza operacionalmente o princípio de minimização de dados preconizado pelo RGPD, constituindo simultaneamente uma vantagem técnica (redução de largura de banda e latência) e uma garantia de conformidade regulatória.

Table 1. Topologia e comunicações do sistema Dance4Life.

Agente	Tipo / Papel	Communicates With
BaseSenderAgent (API)	Gateway REST para ACL	SensorAgent, ClusteringAgent
SensorAgent	Reativo de entrada	BaseSenderAgent, CoordinatorAgent
CoordinatorAgent	Orquestrador de encaminhamento	SensorAgent, HarAgent, DatabaseAgent
HarAgent	Encaminhamento da fase HAR	CoordinatorAgent, EnvironmentAgent
EnvironmentAgent	Enriquecimento ambiental	HarAgent, DatabaseAgent
ClusteringAgent	Processamento de matching social	BaseSenderAgent, CoordinatorAgent
DatabaseAgent	Persistência Firestore	EnvironmentAgent, CoordinatorAgent

2.2 Definição dos Agentes

A implementação atual assenta em duas classes base: BaseBackgroundAgent, utilizada para agentes residentes com ciclo de execução próprio, e BaseSenderAgent, responsável pelo envio de mensagens a partir da API. A primeira integra mecanismos de registo, *heartbeat* periódico e remoção no servidor Flask através dos

behaviours RegisterBehaviour, HeartbeatBehaviour e UnregisterBehaviour, enquanto a segunda encapsula envios pontuais através de SendMessageBehaviour.

O BaseSenderAgent é iniciado no arranque da API e funciona como gateway externo do sistema, enviando mensagens ACL para o SensorAgent, no caso de atividades e recomendações, e para o ClusteringAgent, no caso de operações de matching. Em cada envio são definidos metadados como performative, ontology e conversation-id, sendo o *payload* serializado com jsonpickle. O SensorAgent implementa o behaviour ReceiveSensorDataBehaviour (CyclicBehaviour), recebendo pedidos da API e reencaminhando-os para o CoordinatorAgent com preservação do conversation-id. O agente suporta as ontologias **sensor_activity** e **movement_recommendation** e enquadra-se numa arquitetura reativa [22], uma vez que o seu comportamento assenta em regras diretas de receção e encaminhamento de mensagens. O CoordinatorAgent implementa o behaviour ReceiveCoordinatorDataBehaviour (CyclicBehaviour) e assume o papel de coordenação do *pipeline* principal. Para mensagens associadas à ontologia **sensor_activity**, os dados são encaminhados para o HARAgent; para **movement_recommendation** e matching, os dados seguem diretamente para o DatabaseAgent após um pequeno enriquecimento do *payload*. Apesar do seu funcionamento predominantemente reativo, a lógica de decisão baseada em ontologias e performativas introduz características híbridas [22], permitindo adaptar o encaminhamento em função do contexto da mensagem recebida. O HARAgent implementa o behaviour ReceiveHarDataBehaviour (CyclicBehaviour), encaminhando pedidos relacionados com atividade física para o EnvironmentAgent. Nesta fase do protótipo, o agente não executa ainda inferência HAR baseada em modelos de Machine Learning, funcionando como uma etapa modular preparada para futura integração de modelos CNN (Convolutional Neural Network) ou LSTM (Long Short-Term Memory). Atualmente, o seu comportamento é essencialmente reativo. O EnvironmentAgent implementa o behaviour EnvironmentDataBehaviour (CyclicBehaviour), sendo responsável pelo enriquecimento contextual dos dados. O agente descodifica o *payload* recebido, obtém informação de temperatura através de coordenadas geográficas e adiciona o atributo **weather_temperature** antes de encaminhar os dados para o DatabaseAgent. A necessidade de consultar informação externa confere-lhe características híbridas [22], combinando comportamento reativo com aquisição contextual de conhecimento. O ClusteringAgent implementa o behaviour ReceiveClusteringDataBehaviour (CyclicBehaviour), processando pedidos associados à ontologia matching através do modelo UserActivityClusterModel. O agente é ainda responsável pela publicação de convites para utilizadores através do endpoint `/set_user_match/<user_id>`, encaminhando posteriormente os resultados para o CoordinatorAgent. Entre os agentes implementados, é o que mais se aproxima de uma arquitetura deliberativa [22], uma vez que realiza processamento sobre perfis de utilizadores e dados de localização para suportar decisões de emparelhamento social.

Por fim, o DatabaseAgent implementa o behaviour ReceiveDatabaseDataBehaviour (CyclicBehaviour), removendo o atributo **visited_agents** antes da persistência

dos dados. O agente suporta operações de `save_activity`, `save_movement_recommendation` e `save_matching` em Firestore, de acordo com a ontologia recebida. Na tipologia apresentada por Novais e Analide [22], enquadra-se como um agente reativo orientado a serviços, dedicado a operações de persistência e I/O (*Input/Output*).

Table 2. Síntese dos agentes, behaviours, tipos e classificação arquitetural.

Agente	Behaviour	Tipo	Arquitetura	Ontologia
BaseSenderAgent	SendMessageBehaviour	OneShotBehaviour	Reativo	s_a, m_r, m
SensorAgent	ReceiveSensorDataBehaviour	CyclicBehaviour	Reativo	s_a, m_r
CoordinatorAgent	ReceiveCoordinatorDataBehaviour	CyclicBehaviour	Híbrido	s_a, m_r, m
HARAgent	ReceiveHarDataBehaviour	CyclicBehaviour	Reativo [†]	s_a
EnvironmentAgent	EnvironmentDataBehaviour	CyclicBehaviour	Híbrido	s_a
ClusteringAgent	ReceiveClusteringDataBehaviour	CyclicBehaviour	Deliberativo	m
DatabaseAgent	ReceiveDatabaseDataBehaviour	CyclicBehaviour	Reativo	s_a, m_r, m

s_a = sensor_activity; m_r = movement_recommendation; m = matching
[†] Futuro: Deliberativo após integração de modelo HAR

2.3 Protocolos de Comunicação e Performativas FIPA-ACL

A comunicação entre agentes no Dance4Life segue uma semântica compatível com o padrão FIPA-ACL, suportada pelas capacidades disponibilizadas pelo SPADE. As mensagens trocadas incluem metadados como performative, ontology e conversation-id, permitindo identificar o tipo de operação, o domínio semântico associado e o contexto da conversação. Na implementação atual, as performativas utilizadas no fluxo principal são request e inform. Embora as constantes agree e failure estejam definidas na configuração do sistema, a sua utilização ainda não ocorre de forma consistente em todas as interações. Assim, o sistema implementa parcialmente o protocolo FIPA Request Interaction Protocol, existindo uma fase de iniciação baseada em mensagens request e uma fase de resposta através de mensagens inform, mas sem um mecanismo completo de handshake entre todos os agentes. O modelo de comunicação adotado segue uma sequência relativamente linear. Na fase de iniciação, o agente emissor envia uma mensagem ao próximo agente da cadeia, incluindo a ontologia correspondente ao domínio de operação, como `sensor_activity`, `movement_recommendation` ou `matching`, um conversation-id gerado automaticamente no SendMessageBehaviour, e ainda um *payload* serializado através de jsonpickle. Após receção da mensagem, cada agente procede à decodificação, enriquecimento ou transformação do *payload* e reencaminha os dados para o agente seguinte, preservando os metadados essenciais do contexto da conversação.

Depois de concluir a operação local, seja ela de processamento, encaminhamento ou persistência, o agente responde ao emissor imediato com uma mensagem do tipo inform. Em situações de sucesso, o remetente recebe confirmação da conclusão da operação; em caso de erro, o tratamento é atualmente realizado sobretudo através de mecanismos de exceção e logging local, não existindo

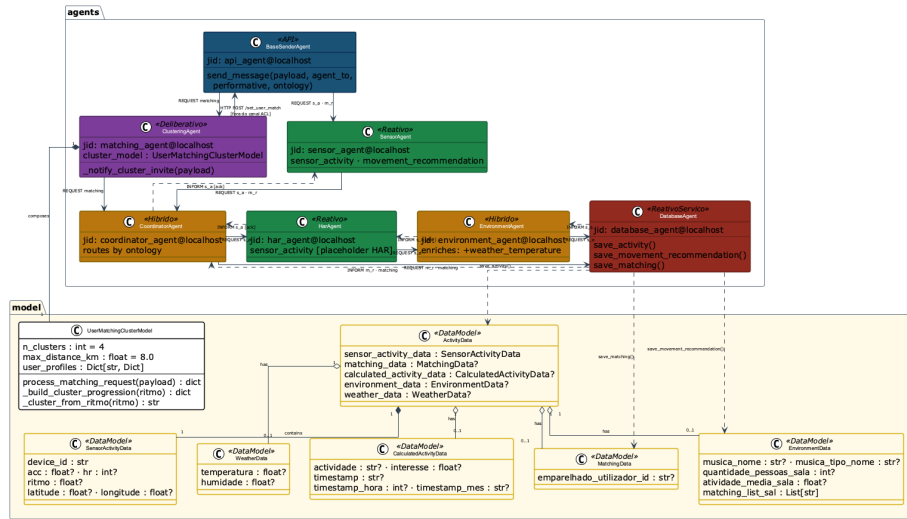


Fig.1. Diagrama de classes AUML do sistema Dance4Life. Pacote **agents**: sete classes de agentes com estereótipos de arquitetura (Reativo, Híbrido, Deliberativo, ReativoServiço, API). Pacote **model**: sete modelos Pydantic ligados por composição e agregação ao ClusteringAgent. Setas sólidas representam mensagens REQUEST (FIPA-ACL) e setas tracejadas representam INFORM (FIPA-ACL); a seta HTTP lateral indica a chamada direta do ClusteringAgent à API fora do canal XMPP.

ainda um protocolo failure uniformemente implementado entre todos os pares de agentes. A gestão de conversações e o isolamento de sessões baseiam-se na utilização de conversation-id únicos, gerados sob a forma de UUIDs (Universally Unique Identifier) pelo BaseSenderAgent. Estes identificadores são propagados ao longo dos diferentes forwards entre agentes, permitindo rastrear integralmente cada cadeia de processamento nos logs e nas mensagens trocadas. Importa, contudo, distinguir entre propagação e isolamento completo de sessões: apesar de o metadado ser preservado ao longo do *pipeline*, o código atual ainda não implementa uma filtragem rigorosa por conversation-id em todos os pontos de receção. Consequentemente, o mecanismo atual garante rastreabilidade e observabilidade do fluxo ponta-a-ponta, facilitando a depuração de interações em cadeias longas e preparando a arquitetura para futuras evoluções orientadas a cenários de maior concorrência e isolamento rigoroso de sessões. Relativamente à troca de dados estruturados, embora o transporte XMPP opere sobre mensagens textuais, o protótipo evita a utilização de strings ad hoc e adota *payloads* estruturados serializados com jsonpickle. Os dados são construídos sob a forma de dicionários estruturados, sendo progressivamente enriquecidos em cada etapa do *pipeline*. Os métodos jsonpickle.encode e jsonpickle.decode asseguram a serialização e desserialização uniforme da informação, reduzindo ambiguidades semânticas e eliminando grande parte da necessidade de parsing manual. Paralelamente, o sistema inclui ainda um modelo de domínio baseado em Pydantic, definido em `model/data_model.py`, destinado à normalização semântica dos dados, embora o *pipeline* atual opere maioritariamente sobre estruturas de dicionários. Esta abordagem cumpre o requisito de privilegiar objetos estruturados em detrimento de mensagens textuais sem esquema definido, contribuindo para uma arquitetura mais robusta, modular e extensível. Em síntese, a implementação atual demonstra alinhamento com os princípios fundamentais do FIPA-ACL, embora ainda existam oportunidades de melhoria ao nível da disciplina protocolar, particularmente na adoção sistemática de **agree/failure** e em mecanismos mais rigorosos de validação semântica e isolamento de sessões.

Table 3. Performativas FIPA-ACL utilizadas no sistema.

Performativa	Direção	Significado	Quando enviada
request	Emissor → Recetor	Pedido de execução/ encaminhamento	Fluxos de atividade, matching e recomendação
inform	Recetor → Emissor	Confirmação local de processamento	Após encaminhamento/ persistência
agree	Recetor → Emissor	Aceitação explícita do pedido	Definida em configuração (não aplicada de forma transversal)
failure	Recetor → Emissor	Resposta semântica de falha	Definida em configuração (não aplicada de forma transversal)

2.4 Diagramas de Atividades AUML por Protocolo

Os três diagramas seguintes representam o comportamento do sistema segundo a notação *Agent UML* (AUML) [22], adoptando pistas de actividade (*swimlanes*) por agente e setas diferenciadas para as performativas REQUEST (encaminhamento) e INFORM (confirmação). Os diagramas foram gerados a partir do

código-fonte verificado e reflectem a implementação efectiva, não um modelo idealizado.

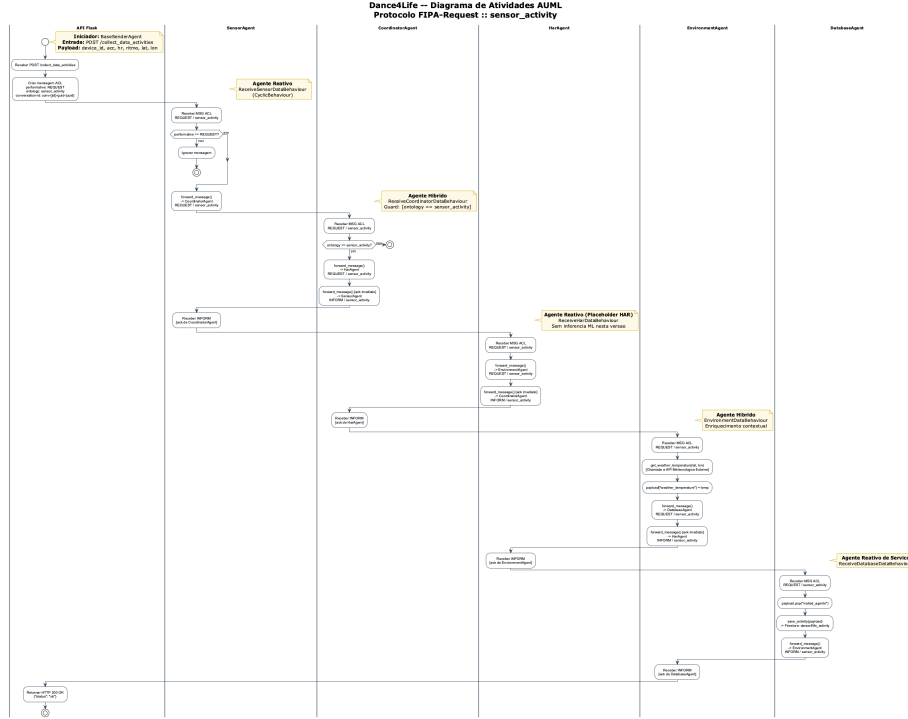


Fig. 2. Diagrama de actividades AUMML — Protocolo FIPA-Request, ontologia **sensor_activity**. Fluxo: API → SensorAgent → CoordinatorAgent → HarAgent → EnvironmentAgent → DatabaseAgent. Retorno INFORM na direcção inversa após cada operação.

2.5 Diagramas de Colaboração AUMML por Protocolo

Os diagramas de colaboração AUMML (também designados *communication diagrams* em UML 2.x) representam os agentes como instâncias nomeadas (no formato `jid:Classe`) e as ligações entre eles como canais de comunicação persistentes, com mensagens ACL numeradas sequencialmente sobre essas ligações [22]. Distinguem-se dos diagramas de actividades por enfatizarem a **estrutura da rede de comunicação** e a **ordem das trocas de mensagens**, em vez do fluxo de controlo interno de cada agente. As setas sólidas correspondem à performativa REQUEST (encaminhamento), as setas tracejadas à performativa INFORM (confirmação de processamento) e as setas duplas indicam chamadas HTTP externas fora do canal ACL.

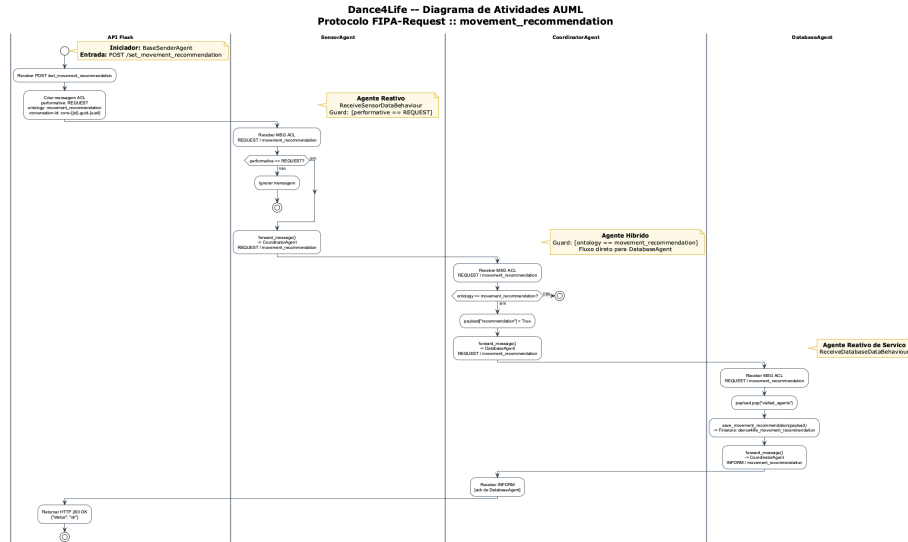


Fig. 3. Diagrama de atividades AUML — Protocolo FIPA-Request, ontologia `movement_recommendation`. Fluxo simplificado: API → SensorAgent → CoordinatorAgent → DatabaseAgent para CoordinatorAgent. Retorno INFORM de DatabaseAgent para CoordinatorAgent.

2.6 Diagramas de Sequência AUML por Protocolo

Os diagramas de sequência AUML representam os agentes como linhas de vida (*lifelines*) identificadas pelo par `jid:Classe<<estereótipo>>` e as mensagens ACL como setas numeradas sobre o eixo temporal (de cima para baixo) [22]. Relativamente aos diagramas de colaboração, acrescentam três elementos essenciais: os **blocos de ativação** (rectângulos sobre a lifeline, que delimitam o período de processamento de cada agente), as **chamadas internas** (auto-mensagens, que representam operações locais como enriquecimento contextual ou persistência em Firestore) e as **secções de protocolo** (bandas que separam a Iniciação FIPA-Request, o Processamento e a Resolução). As setas sólidas com ponta ($\rightarrow$) correspondem à performativa REQUEST e as setas tracejadas ($-->$) à performativa INFORM.

2.7 Statecharts: Ciclo de Vida e Pipeline de Mensagens

O diagrama 11 modela o ciclo de vida do par servidor Flask – `BaseBackgroundAgent`: arranque, registo via `POST /register_agent`, monitorização periódica por *heartbeat* (period = 10s), deteção de *timeout* (`HEARTBEAT_TIMEOUT = 30s`) pela *cleanup thread* do servidor, e paragem ordenada com `POST /unregister_agent`.

O diagrama 12 apresenta os estados internos de cada agente e as transições inter-agentes para a ontologia `sensor_activity`, a cadeia mais completa do sistema. Cada agente alterna entre *Aguardando* (`receive(timeout=10)`),

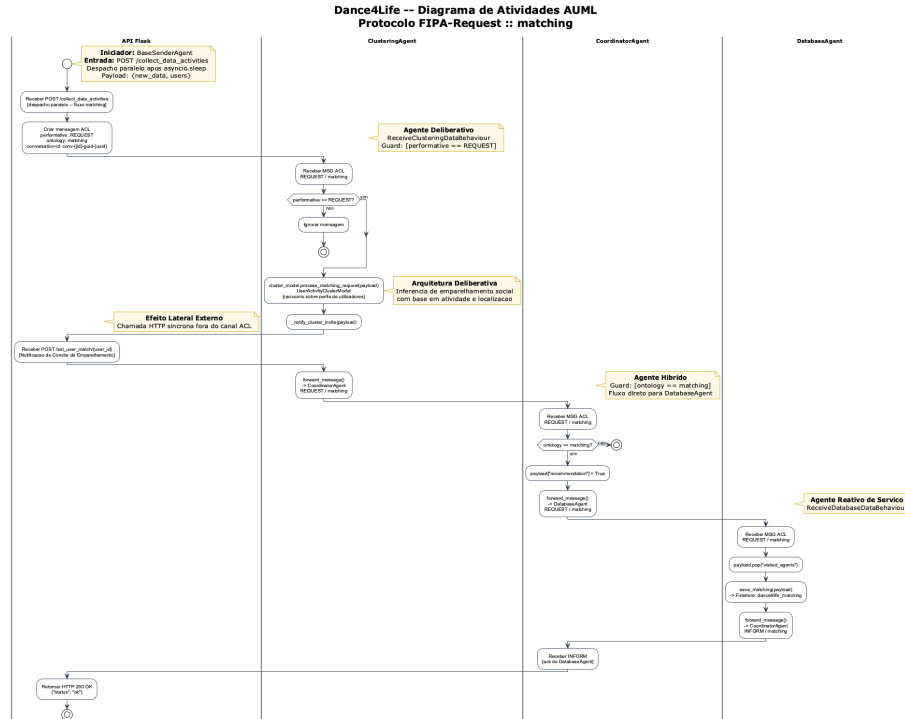


Fig.4. Diagrama de atividades AUMIL — Protocolo FIPA-Request, ontologia matching. Fluxo: API → ClusteringAgent (inferência deliberativa) → notificação HTTP lateral (/set_user_match) → CoordinatorAgent → DatabaseAgent. Retorno INFORM de DatabaseAgent para CoordinatorAgent.

Dance4Life -- Diagrama de Colaboracao AUML Protocolo FIPA-Request :: *sensor_activity*

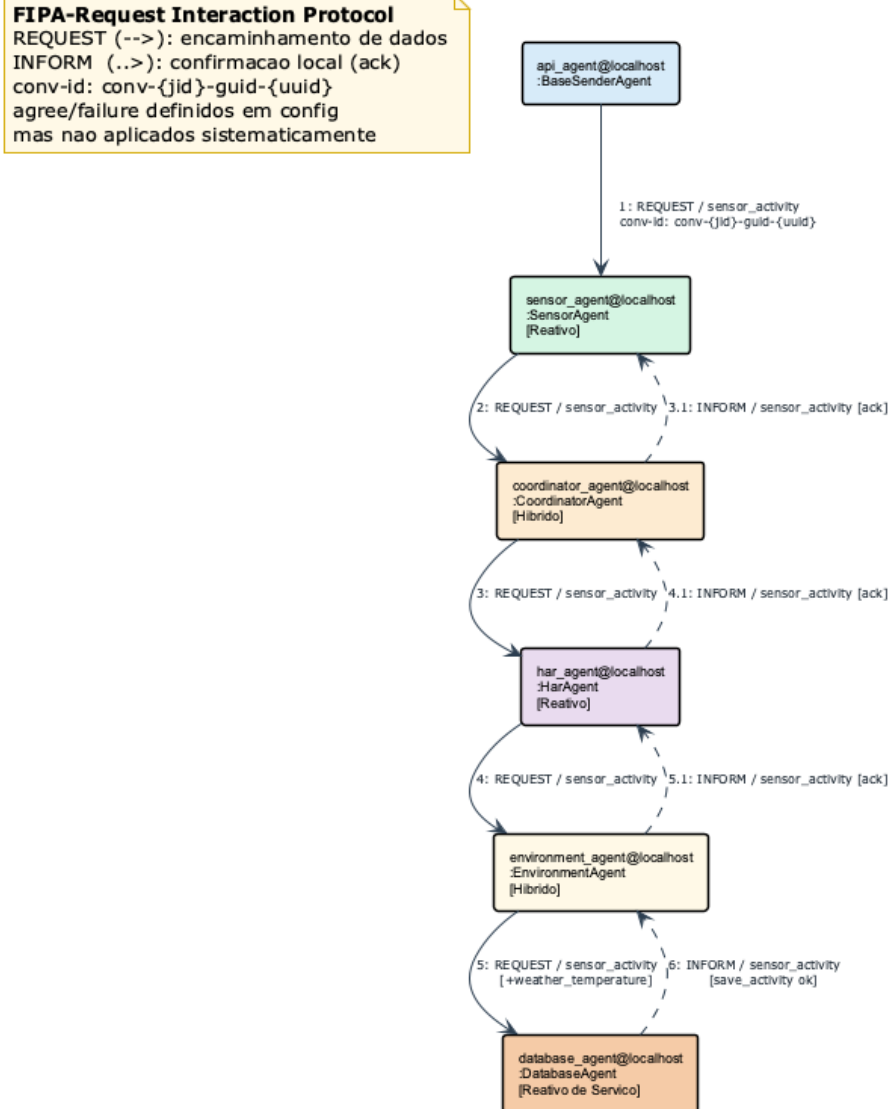


Fig. 5. Diagrama de colaboração AUML — Protocolo FIPA-Request, ontologia *sensor_activity*. Mensagens REQUEST numeradas 1–5 (setas sólidas, esquerda para a direita); mensagens INFORM 3.1, 4.1, 5.1 e 6 (setas tracejadas, direita para a esquerda). O conv-id é propagado em todas as mensagens da conversação.

Dance4Life -- Diagrama de Colaboracao AUMl Protocolo FIPA-Request :: *movement_recommendation*

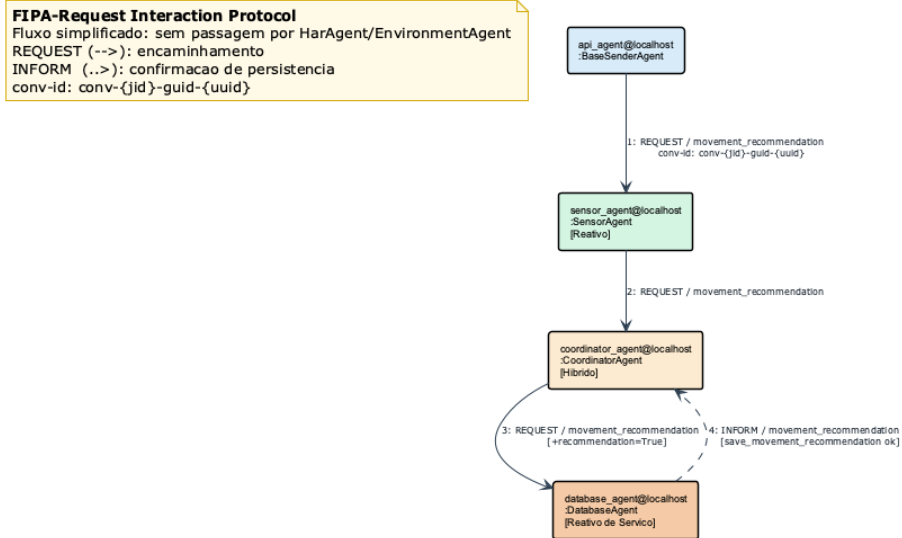


Fig. 6. Diagrama de colaboração AUMl — Protocolo FIPA-Request, ontologia *movement_recommendation*. Fluxo simplificado de quatro mensagens: REQUEST 1–3 (API → SensorAgent → CoordinatorAgent → DatabaseAgent) e INFORM 4 de confirmação de persistência.

Dance4Life -- Diagrama de Colaboracao AUML Protocolo FIPA-Request :: *matching*

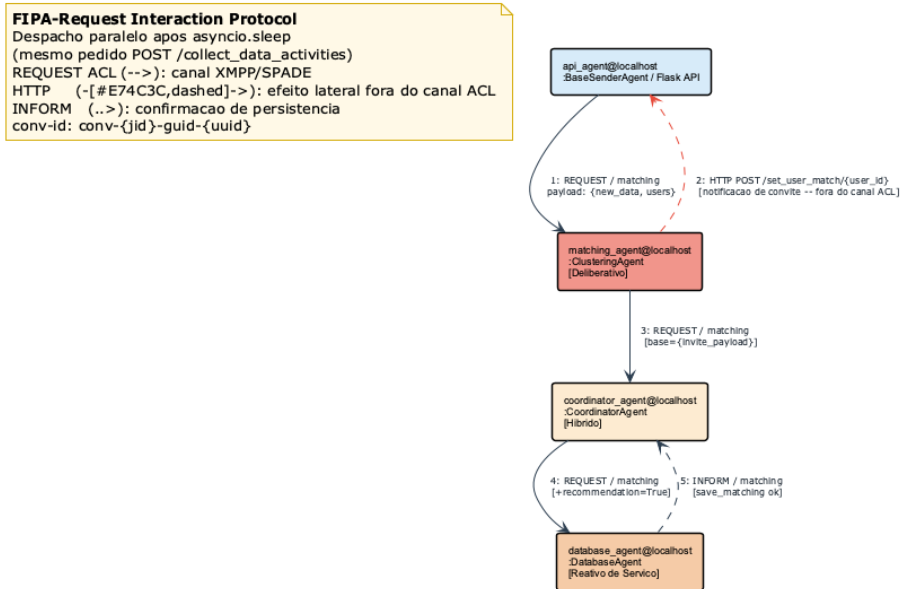


Fig. 7. Diagrama de colaboração AUML — Protocolo FIPA-Request, ontologia *matching*. A mensagem 2 representa o efeito lateral HTTP (POST /set_user_match/{id}) que o ClusteringAgent efectua directamente sobre a Flask API fora do canal ACL, antes de encaminhar o resultado via XMPP para o CoordinatorAgent (mensagem 3).

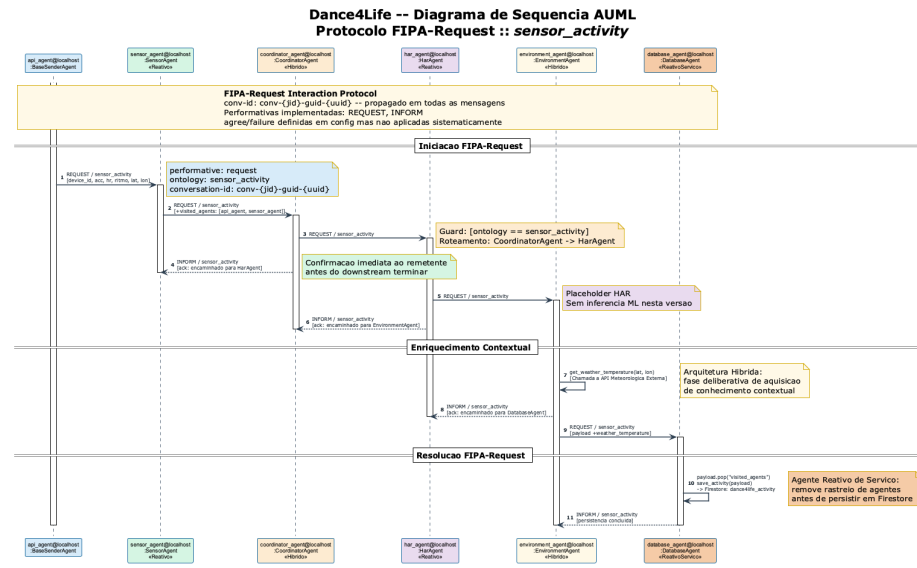


Fig.8. Diagrama de sequência AUMI — Protocolo FIPA-Request, ontologia *sensor_activity*. 11 mensagens numeradas: REQUEST 1–3, 5, 9 (setas sólidas); INFORM 4, 6, 8, 11 (setas tracejadas); auto-mensagens 7 (*get_weather_temperature*) e 10 (*save_activity*). Os blocos de activação mostram o período em que cada agente aguarda resposta do downstream.

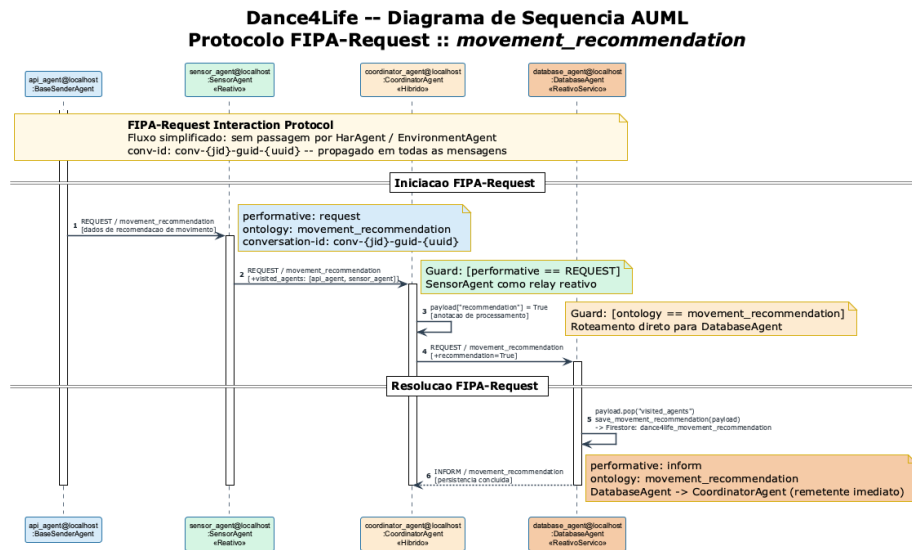


Fig. 9. Diagrama de sequência AUMI — Protocolo FIPA-Request, ontologia `movement_recommendation`. Fluxo de 6 mensagens: REQUEST 1-2 (API→SensorAgent→CoordinatorAgent), auto-mensagem 3 (payload["recommendation"]=True), REQUEST 4 (→DatabaseAgent), auto-mensagem 5 (save_movement_recommendation) e INFORM 6 de confirmação.

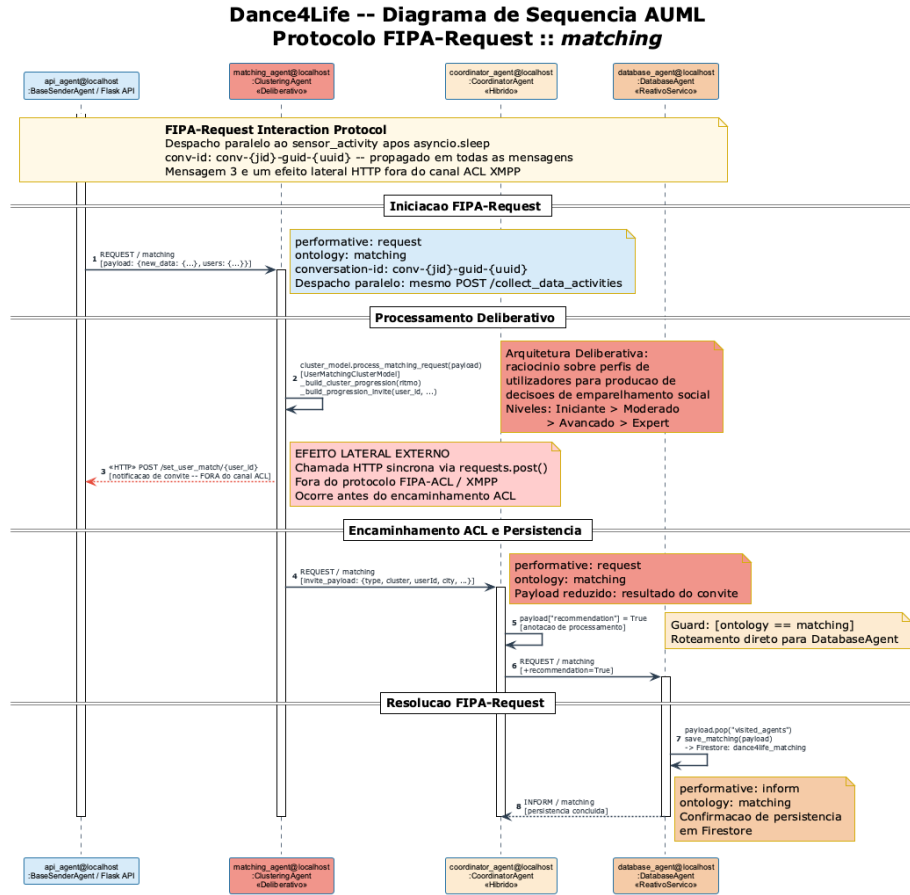


Fig.10. Diagrama de sequência AUML — Protocolo FIPA-Request, ontologia matching. A mensagem 3 (HTTP POST /set_user_match/{id}) é um efeito lateral síncrono executado pelo ClusteringAgent **fora do canal ACL** antes de retomar o protocolo FIPA-Request na mensagem 4. O processamento deliberativo do UserMatchingClusterModel é representado como auto-mensagem 2.

Processando (decodificação e verificação) e *Encaminhando* (`forward_message`), emitindo um INFORM de reconhecimento ao remetente após cada salto.

3 Implementação e Funcionamento (Baseado em SPADE)

O sistema Dance4Life foi implementado em Python e SPADE, com execução assíncrona sobre `asyncio` e comunicação ACL sobre XMPP. A solução combina uma API Flask para interação externa com uma rede de agentes residentes em loops próprios (`threads` dedicadas + `event loop` por agente), permitindo desacoplamento entre entrada HTTP e processamento multiagente.

3.1 Instalação Externa da SPADE e Ambiente de Execução

A SPADE é uma *framework* Python para o desenvolvimento de agentes assíncronos baseados em XMPP, alinhada com os princípios de comunicação FIPA-ACL [21]. Ao contrário de plataformas como o JADE (Java Agent DEvelopment Framework) [22], desenvolvido em Java, o SPADE tira partido do motor assíncrono `asyncio` do Python, tornando-o particularmente adequado para ambientes de prototipagem rápida. No contexto deste projeto, a SPADE não foi integrada no *stack* Android; foi instalada externamente num ambiente Python autónomo localizado na pasta Python do projeto, com gestão de dependências em `pyproject.toml`. As dependências relevantes incluem SPADE, Flask, `jsonpickle`, `firebase-admin`, `requests` e `pandas`. A execução segue o ciclo típico:

```
python -m venv .venv
source .venv/bin/activate
pip install -e .
python main.py
```

3.2 Fluxo de Dados — Pipeline Sequencial Assíncrono

Apesar de os Sistemas Multiagente serem concorrentes por natureza, o protótipo implementado adota um *pipeline* sequencial de processamento. No caso da ontologia `sensor_activity`, o fluxo inicia-se na API, através do `BaseSenderAgent`, que envia uma mensagem `request` para o `SensorAgent`. Este agente reencaminha a informação para o `CoordinatorAgent`, que por sua vez encaminha os dados para o `HARAgent`. Após a fase de interpretação da atividade, o `HARAgent` envia o *payload* para o `EnvironmentAgent`, responsável por enriquecer os dados com informação adicional, nomeadamente o atributo `weather_temperature` obtido através de sensores externos. Finalmente, o `EnvironmentAgent` encaminha os dados para o `DatabaseAgent`, que realiza a persistência no Firebase e devolve uma mensagem `inform` ao emissor imediato. Para a ontologia `movement_recommendation`, o fluxo segue uma sequência mais reduzida, correspondente ao percurso `API → SensorAgent → CoordinatorAgent → DatabaseAgent`. Já no caso da ontologia `matching`, a API comunica diretamente com o `ClusteringAgent`, responsável pelo

processamento de convites e pela lógica de agrupamento social, encaminhando posteriormente os dados para o CoordinatorAgent e para o DatabaseAgent. A assíncronia do sistema resulta da utilização de behaviours cíclicos e da existência de event loops independentes em cada agente. Esta abordagem permite que os diferentes componentes executem processamento concorrente sem bloquearem a thread principal da API, garantindo maior capacidade de resposta perante múltiplos pedidos simultâneos.

3.3 Gestão de Conversações e Isolamento de Mensagens

A coordenação de interações distribuídas entre múltiplos agentes requer mecanismos que permitam identificar, acompanhar e distinguir cada fluxo de processamento ao longo do sistema. Para esse efeito, o Dance4Life utiliza o atributo conversation-id, previsto na especificação FIPA-ACL, como identificador único de cada conversa o iniciada na plataforma. Sempre que a API necessita de desencadear uma nova opera o, o BaseSenderAgent gera automaticamente um identificador  nico recorrendo   biblioteca uuid, associando-o   mensagem ACL antes do envio. Este identificador   posteriormente propagado ao longo de todos os encaminhamentos efetuados pelos diferentes agentes do *pipeline*.

```
# BaseSenderAgent.SendMessageBehaviour
msg.set_metadata("conversation-id", f"conv-{self.agent.jid}-
guid-{uuid.uuid4()}")
```

A preserva o deste metadado durante os diferentes forwards permite manter a identidade l gica da conversa o independentemente do n mero de agentes envolvidos. Desta forma, torna-se poss vel rastrear integralmente o percurso de cada pedido desde a sua origem na API at    persist ncia final dos dados no DatabaseAgent. Do ponto de vista operacional, o uso de conversation-id oferece benef cios importantes ao n vel da observabilidade e depura o do sistema. Em cen rios distribuídos, onde m ltiplas mensagens podem circular simultaneamente entre v rios agentes, a exist ncia de um identificador comum permite correlacionar eventos registados nos logs, facilitando a an lise de falhas, a monitoriza o do desempenho e a reconstru o do hist rico de execu o de cada pedido. Contudo, importa distinguir entre rastreabilidade e isolamento efetivo de sess es. Embora o identificador seja atualmente propagado entre os agentes, a implementa o ainda n o realiza uma valida o sistem tica do conversation-id em todos os pontos de rece o. Assim, os agentes utilizam o identificador sobretudo para efeitos de monitoriza o e contexto, n o existindo ainda mecanismos completos de correla o de respostas ou filtra o rigorosa de mensagens pertencentes a diferentes sess es concorrentes. Em cen rios de elevada concorr ncia, esta limita o poder  originar dificuldades na associa o inequ voca entre pedidos e respostas, especialmente quando m ltiplas intera  es partilham a mesma ontologia e performativa. Como trabalho futuro, recomenda-se a implementa o de mecanismos formais de correla o baseados em conversation-id, complementados pelos atributos ACL reply-with e in-reply-to, permitindo garantir isolamento completo das conversa  es e maior robustez na coordena o distribu da.

Em síntese, a utilização de identificadores únicos de conversação constitui um primeiro passo para a gestão estruturada das interações entre agentes, assegurando rastreabilidade ponta-a-ponta e preparando a arquitetura para futuras evoluções orientadas à escalabilidade, concorrência e conformidade protocolar com os princípios FIPA-ACL.

3.4 Troca de Mensagens Baseada em Objetos (Pydantic + Jsonpickle)

A troca de mensagens no sistema evita a utilização de texto não estruturado e adota um paradigma de serialização orientado a objetos. Para esse efeito, o `jsonpickle` é utilizado para serializar *payloads* Python no corpo das mensagens ACL, permitindo que os dados sejam transmitidos de forma estruturada entre agentes. No lado do recetor, o mesmo mecanismo é responsável pela descodificação dos *payloads* antes de qualquer processamento ou transformação adicional. Ao longo do *pipeline*, os diferentes agentes enriquecem progressivamente a informação com dados contextuais, como `visited_agents` ou `weather_temperature`, preservando sempre uma estrutura consistente dos objetos transmitidos. Paralelamente, os modelos Pydantic definidos em `model/data_model.py` fornecem suporte para uma futura evolução do sistema em direção a validação tipada ponta-a-ponta. Esta abordagem contribui significativamente para a legibilidade, manutenção e qualidade semântica da comunicação distribuída entre agentes.

3.5 Validação de Ontologias

As ontologias suportadas são definidas em configuração central (`AgentOntologies`): `sensor_activity`, `movement_recommendation` e `matching`. Os agentes executam decisões condicionais por ontologia e performativa para encaminhar dados ao próximo componente. No estado atual, a validação ocorre por ramos condicionais em cada `behaviour`; não existe ainda um bloco transversal de rejeição semântica com resposta `failure` padronizada para todos os casos. Esta limitação é relevante para evolução da robustez protocolar.

Exemplo simplificado do padrão usado:

```
if ontology == AgentOntologies.SENSOR_ACTIVITY:
    if performative == AgentPerformatives.REQUEST:
        await self.agent.forward_message(...)
```

3.6 Estrutura de Ficheiros do Projeto

A estrutura de ficheiros do protótipo Dance4Life reflete uma arquitetura modular que segue as boas práticas de desenvolvimento em Python, promovendo a separação de responsabilidades através de uma organização das suas diretorias. O projeto encontra-se dividido em pastas específicas para isolar os comportamentos assíncronos e as definições das classes dos agentes, os modelos de validação de dados, os serviços de persistência na cloud, as configurações globais e

as interfaces de fronteira baseadas no protocolo REST. No centro desta organização encontra-se a diretoria `agents/`, que contém o núcleo comportamental do ecossistema. A classe abstrata `BaseBackgroundAgent` está centralizada no ficheiro `base_background_agent.py`, sendo responsável por gerir o ciclo de vida dos agentes através da criação de um loop de eventos dedicado (`agent_loop`) executado numa thread secundária em segundo plano. O gateway lógico entre as rotas HTTP e a rede XMPP é assegurado pela classe `BaseSenderAgent`, localizada em `base_sender_agent.py`, que permite a conversão e o envio síncrono de mensagens através do método de bloqueio controlado `join()`. Os restantes ficheiros desta diretoria encapsulam os agentes especializados do *pipeline*: o `sensor_agent.py` define o agente de entrada reativa que ingere os dados físicos do edge ; o `coordinator_agent.py` contém o orquestrador híbrido encarregue de triar as mensagens por ontologia ; o `har_agent.py` serve como etapa modular para o reconhecimento de atividades humanas ; o `environment_agent.py` implementa o enriquecimento do *payload* com dados meteorológicos ; o `matching_agent.py` aloja o `ClusteringAgent` focado na lógica de emparelhamento social ; e o `database_agent.py` gere a persistência documental. As restantes pastas fornecem o suporte de infraestrutura e dados necessário para o funcionamento do sistema. A diretoria `model/` isola a lógica de dados, contendo o ficheiro `data_model.py` com os esquemas de validação estrutural do Pydantic, e o ficheiro `user_matching_cluster.py`, que implementa o algoritmo de agrupamento `UserActivityClusterModel` consumido pelo agente de matching. A pasta `services/` aloja o ficheiro `firebase_service.py`, que centraliza as rotas assíncronas de escrita em Firestore (`save_activity`, `save_matching` e `save_movement_recommendation`). Os módulos de integração com APIs externas de meteorologia encontram-se isolados em `sensor/external_sensors.py`, garantindo que as classes dos comportamentos fiquem livres de código de infraestrutura de rede. Por fim, a configuração central do sistema é gerida em `config/config.py`, onde se parametrizam as constantes de JIDs, credenciais, e os enumerados de ontologias e performativas FIPA-ACL, evitando o acoplamento rígido de strings e garantindo a consistência semântica de todo o ecossistema distribuído.

3.7 Monitorização de Agentes — Heartbeat e Registo

O sistema Dance4Life assenta numa abordagem multiagente baseada na plataforma SPADE, onde diferentes agentes colaboram para processar dados de atividade física, informação ambiental e recomendações de emparelhamento. Para garantir a disponibilidade e a correta coordenação destes componentes, foi implementado um mecanismo de monitorização baseado em *heartbeats*. Todos os agentes do sistema herdam da classe base `BaseBackgroundAgent`, responsável por disponibilizar funcionalidades comuns relacionadas com o ciclo de vida dos agentes. Durante a sua inicialização, cada agente efetua automaticamente o registo junto do `WebService` através do endpoint `/register_agent`, ficando disponível para participar nas operações do sistema. Após o registo, é iniciado um comportamento periódico designado por `HeartbeatBehaviour`, responsável pelo envio de

mensagens de *heartbeat* para o endpoint */heartbeat*. Estas mensagens são enviadas a cada 10 segundos e contêm o identificador único do agente (JID – Jabber ID), permitindo ao servidor confirmar que o agente permanece operacional. No lado do servidor, é mantida uma estrutura de registo contendo todos os agentes ativos e o instante temporal do último *heartbeat* recebido. Sempre que um *heartbeat* é recebido, o campo *last_seen* associado ao respetivo agente é atualizado, permitindo acompanhar o estado de disponibilidade de cada componente da plataforma. De forma complementar, o servidor executa uma tarefa periódica de limpeza responsável por verificar os tempos de inatividade dos agentes registados. Caso um agente deixe de enviar *heartbeats* durante um período superior a 30 segundos, é considerado indisponível e removido automaticamente da lista de agentes ativos. Este mecanismo permite detetar falhas de comunicação, encerramentos inesperados ou interrupções de serviço sem necessidade de intervenção manual. Quando um agente termina a sua execução de forma controlada, é ainda executado um processo de remoção através do endpoint */unregister_agent*, garantindo que o registo é removido imediatamente e evitando a permanência de informação desatualizada no sistema. A implementação deste mecanismo contribui para a robustez da arquitetura Dance4Life, assegurando que apenas agentes ativos participam nos processos de comunicação e processamento. Adicionalmente, esta abordagem estabelece uma base para futuras extensões relacionadas com tolerância a falhas, recuperação automática de componentes e gestão dinâmica da infraestrutura multiagente.

4 Resultados e Análise Crítica

A implementação do protótipo Dance4Life permitiu validar a viabilidade técnica de uma arquitetura multiagente integrada com API Flask, SPADE e Firebase. Os testes funcionais realizados sobre os endpoints de entrada e os diferentes fluxos entre agentes demonstraram o correto processamento de eventos de atividade, recomendações de movimento e operações de matching social.

Apesar dos resultados obtidos, a análise do sistema evidencia algumas limitações ao nível da robustez protocolar, validação semântica e gestão de concorrência, identificando igualmente oportunidades de melhoria para futuras evoluções da arquitetura.

4.1 Sumário do Funcionamento e Testes

Durante as sessões de teste e execução, o comportamento emergente da rede de agentes confirmou o correto funcionamento do sistema com as regras de negócio previamente desenhadas em Agent UML. Verificou-se, em primeiro lugar, o correto encadeamento funcional das atividades, em que os dados recebidos através do endpoint */collect_data_activities* percorrem sucessivamente o SensorAgent, CoordinatorAgent, HARAgent, EnvironmentAgent e DatabaseAgent, culminando na persistência da informação na coleção *dance4life_activity*. Paralelamente, o fluxo de matching revelou-se operacional, sendo os pedidos pro-

cessados pelo ClusteringAgent, responsável pela emissão de convites para utilizadores e pela persistência final dos dados na coleção `dance4life_matching`.

Os testes permitiram ainda validar a persistência distribuída em múltiplas coleções, assegurada pelo DatabaseAgent, que grava dados distintos relacionados com atividade, recomendações e matching, refletindo uma clara separação de responsabilidades baseada nas diferentes ontologias do sistema. Adicionalmente, a comunicação orientada a *payloads* estruturados demonstrou vantagens significativas ao longo do *pipeline*, uma vez que a utilização de jsonpickle eliminou a necessidade de parsing textual manual e garantiu consistência semântica dos atributos trocados entre agentes. Por fim, o mecanismo de monitorização baseado em operações de register, *heartbeat* e unregister implementado no servidor API forneceu visibilidade operacional sobre o estado e disponibilidade dos agentes ativos na plataforma.

4.2 Análise Crítica — Estrangulamento Sequencial

Apesar dos resultados positivos alcançados, a análise crítica do sistema revela a existência de limitações estruturais que deverão ser abordadas em futuras evoluções, de forma a garantir maiores níveis de escalabilidade, resiliência e extensibilidade da arquitetura. Uma das principais limitações relaciona-se com a conformidade apenas parcial com o padrão FIPA-ACL. Embora as performativas agree e failure estejam definidas na configuração do sistema, a implementação atualmente operacional utiliza predominantemente mensagens request e inform, não existindo uma aplicação uniforme de mecanismos de handshake semântico completo, nomeadamente aceitação explícita e comunicação protocolar de falhas.

Outra fragilidade identificada prende-se com a validação de ontologias, que é atualmente realizada através de blocos condicionais específicos em cada agente. Esta abordagem não dispõe ainda de um mecanismo transversal e estruturado para rejeição de ontologias desconhecidas, o que pode dificultar a robustez e consistência do sistema em cenários de expansão funcional.

Paralelamente, o encadeamento rígido dos agentes constitui também uma limitação relevante, uma vez que os endereços dos agentes se encontram hard-coded e o *pipeline* depende de uma sequência fixa de forwards. A ausência de mecanismos de descoberta dinâmica reduz a flexibilidade da arquitetura e limita a escalabilidade e substituição transparente de serviços.

Ao nível funcional, o HARAgent permanece ainda numa fase preliminar, funcionando essencialmente como etapa de encaminhamento de dados. A componente de inferência propriamente dita, baseada em modelos treinados de reconhecimento de atividade humana, constitui trabalho futuro. Adicionalmente, embora o sistema gere e propague conversation-id ao longo das interações, não existe ainda um mecanismo rigoroso de correlação e filtragem de respostas capaz de garantir isolamento completo das sessões em cenários de elevada concorrência.

Foram igualmente identificadas fragilidades ao nível da robustez operacional. Atualmente, os erros de processamento são tratados sobretudo através de exceções e mecanismos de logging local, não existindo ainda políticas uniformes de retries automáticos, gestão semântica de timeouts ou estratégias de compensação

transacional. Estas limitações reduzem a tolerância a falhas e a capacidade de recuperação automática do sistema perante problemas de comunicação ou processamento distribuído.

Em síntese, o protótipo cumpre os objetivos nucleares associados à implementação de uma arquitetura distribuída baseada em agentes e demonstra funcionamento end-to-end dos principais fluxos operacionais.

Contudo, para atingir um nível mais elevado de maturidade acadêmica e técnica, recomenda-se reforçar a disciplina protocolar associada ao padrão FIPA-ACL, bem como introduzir mecanismos mais robustos de resiliência, coordenação e gestão de concorrência distribuída.

5 Conclusões e Recomendações para Melhoria

Apresentam-se as principais conclusões obtidas durante o desenvolvimento do protótipo Dance4Life, bem como uma análise das limitações identificadas ao longo da implementação. São ainda discutidas possíveis evoluções futuras da arquitetura, com foco no reforço da robustez, escalabilidade e conformidade protocolar do sistema.

5.1 Conclusões

O desenvolvimento do protótipo Dance4Life demonstra a viabilidade de uma infraestrutura baseada em Sistemas Multiagente, implementada em Python com recurso ao SPADE, aplicada ao domínio do envelhecimento ativo. O sistema evidencia uma integração funcional entre uma API externa, uma cadeia de agentes distribuídos e os mecanismos de persistência em cloud, apresentando simultaneamente uma arquitetura modular, clara separação de responsabilidades e capacidade de evolução através da introdução de novos agentes e ontologias.

Do ponto de vista dos objetivos definidos no enunciado, os resultados alcançados podem ser considerados globalmente positivos. Foi concebida e implementada uma arquitetura distribuída funcional, suportada por diferentes classes de agentes e behaviours operacionalizados em SPADE. A comunicação entre agentes recorre a metadados compatíveis com ACL, incluindo ontologias explicitamente definidas, permitindo estruturar semanticamente os fluxos de interação.

Paralelamente, o sistema utiliza *payloads* estruturados serializados com jsonpickle, contribuindo para maior consistência semântica e redução da complexidade associada ao parsing manual de mensagens. Os resultados produzidos pelos diferentes agentes são ainda persistidos em Firebase, assegurando armazenamento distribuído e suporte à continuidade operacional da plataforma.

Contudo, a conformidade com o padrão FIPA-ACL permanece ainda parcial, sobretudo ao nível das performativas utilizadas. Apesar de o sistema implementar interações baseadas em request e inform, não existe ainda um mecanismo completo de handshake protocolar suportado por performativas como agree e failure. Esta limitação não compromete a validade da prova de conceito nem os objetivos centrais do projeto, mas estabelece prioridades claras para futuras evoluções técnicas e académicas do sistema.

5.2 Recomendações e Trabalho Futuro

Para reduzir a latência e aumentar o throughput do sistema, recomenda-se a adoção de mecanismos de paralelismo controlado entre ramos independentes do *pipeline*, particularmente em situações em que determinadas operações não apresentam dependência direta entre si, como a análise HAR e o enriquecimento contextual. Esta abordagem poderá ser implementada recorrendo a funcionalidades assíncronas do *asyncio*, nomeadamente através de *asyncio.gather*, complementadas por políticas explícitas de *timeout* e gestão de exceções. Desta forma, o sistema poderá executar múltiplos pedidos em paralelo, aumentando a eficiência global da infraestrutura distribuída.

Outra evolução fundamental passa pelo reforço da conformidade protocolar com FIPA-ACL. Em trabalhos futuros, recomenda-se a implementação integral do ciclo *request* → *agree* → *inform/failure* em todos os pares comunicantes, assegurando uma semântica de interação mais rigorosa e consistente. Paralelamente, deverão ser introduzidos templates de validação baseados em ontologias e mecanismos uniformes de resposta semântica a erros, aumentando a robustez e previsibilidade das interações entre agentes.

A arquitetura atual recorre ainda a endereços estáticos hardcoded, como `har_agent@localhost`, o que limita a flexibilidade e escalabilidade do sistema. Nesse sentido, uma evolução importante consiste na introdução de mecanismos de descoberta dinâmica de serviços através de um Directory Facilitator ou Service Facilitator. Com esta abordagem, os agentes poderiam registar-se dinamicamente e o CoordinatorAgent passaria a descobrir os serviços adequados em tempo de execução, reduzindo o acoplamento estático da arquitetura. Complementarmente, recomenda-se a inclusão de novos agentes especializados. Um Security Agent poderia ser responsável pela auditoria de acessos e monitorização contínua de indicadores fisiológicos, como a frequência cardíaca, tendo autonomia para interromper sessões de dança em caso de deteção de fadiga ou risco clínico. Por sua vez, um Feedback Agent permitiria estabelecer um ciclo de interação proativa com o utilizador, fornecendo recomendações inteligentes, como sugestões musicais personalizadas baseadas no histórico de utilização e nos dados previamente armazenados.

Considerando a natureza sensível dos dados clínicos e demográficos manipulados pela plataforma, torna-se igualmente necessário reforçar os mecanismos de segurança e privacidade em conformidade com o RGPD. Neste contexto, recomenda-se, a adoção de paradigmas de Edge Computing, transferindo parte do processamento cognitivo, incluindo modelos de Machine Learning atualmente emulados no HAR, para o próprio dispositivo do utilizador. Desta forma, apenas parâmetros anonimizados seriam transmitidos pela rede XMPP, evitando a circulação de sinais biométricos brutos. Paralelamente, a comunicação entre agentes deverá incorporar mecanismos de encriptação end-to-end em todos os behaviours de rede. Outra evolução particularmente relevante consiste na implementação de estratégias de Federated Learning, permitindo que os agentes partilhem apenas parâmetros dos modelos treinados, preservando a privacidade dos dados individuais dos utilizadores.

Durante a análise do código existente foram ainda identificadas várias melhorias de implementação que poderão aumentar significativamente a qualidade técnica do sistema. Entre elas destaca-se a normalização da validação de ontologias e performativas através de funções utilitárias transversais no BaseBackgroundAgent, reduzindo redundância e aumentando consistência. Recomenda-se também a introdução de mecanismos robustos de correlação de mensagens com base em conversation-id e reply-to em todos os recetores, garantindo melhor isolamento de sessões em cenários concorrentes.

Outra melhoria importante passa pela eliminação de blocos repetidos de forward, substituindo-os por factories de behaviour parametrizáveis, promovendo reutilização e manutenção mais eficiente do código. Sugere-se ainda a migração gradual dos *payloads* críticos para modelos Pydantic diretamente integrados no caminho de execução, em vez de serem utilizados apenas como contratos de referência. Por fim, torna-se essencial implementar testes automáticos de integração para os três fluxos principais do sistema: atividade, matching e recomendação.

Em síntese, o Dance4Life constitui uma prova de conceito sólida que evidencia o potencial dos Sistemas Multiagente no apoio ao envelhecimento ativo. A arquitetura proposta apresenta uma organização clara, modular e extensível, enquanto a implementação em SPADE demonstra a capacidade dos agentes operarem de forma distribuída e assíncrona. As recomendações identificadas ao longo da análise estabelecem um roteiro concreto para evoluir o protótipo para uma solução mais escalável, inteligente, resiliente e alinhada com as necessidades reais da população sénior.

References

1. Bugaychenko, D., Dzuba, A.: Musical Recommendations and Personalization in a Social Network. In: Proc. RecSys '13, pp. 273–280. ACM (2013)
2. Camarinha-Matos, L.M., Castolo, O., Rosas, J.: A Multi-agent Based Platform for Virtual Communities in Elderly Care. In: IEEE ETFA (2002/2003)
3. Crista, V., Martinho, D.V., Marreiros, G.: Chat4Elderly: A Multi-Agent System for Personalized Wellness Using Generative AI and Wearable Technology. In: Proc. AAMAS 2025
4. Esmaeili, A., Ghorrati, Z., Matson, E.T.: Distributed Agent-Based Collaborative Learning in Cross-Individual Wearable Sensor-Based Human Activity Recognition. arXiv:2311.04236 (2023)
5. Ferri-Molla, I., Linares-Pellicer, J., et al.: Multi-Agent AI System for Adaptive Cognitive Training in Elderly Care. In: Proc. ICAART 2025
6. Fraile, J.A., Bajo, J., Corchado, J.M.: Multi-Agent Architecture for Dependent Environments. *Inteligencia Artificial* **13**(41) (2009)
7. Fraile Nieto, J.A., et al.: The THOMAS Architecture: A Case Study in Home Care Scenarios. Springer (2010)
8. Humayun, M., Jhanjhi, N.Z., et al.: Agent-Based Medical Health Monitoring System. *Sensors* **22**(8), 2820 (2022)
9. Ivaşcu, T., Negru, V.: Activity-Aware Vital Sign Monitoring Based on a Multi-Agent Architecture. *Sensors* **21**(12), 4181 (2021)

10. Mendes, S., Queiroz, J., Leitão, P.: Data Driven Multi-agent m-Health System to Characterize the Daily Activities of Elderly People. In: Proc. CISTI 2017
11. Özkara, M., Turan, M.: Personalized News Recommendation System. *Journal of Technology and Applied Sciences* (2022)
12. Paraschiv, E.A., Vevera, A.V., et al.: Towards Trustworthy AI Agents in Geriatric Medicine. *Future Internet* **18**, 75 (2026)
13. Sánchez San Blas, H., Mendes, A.S., et al.: A Multi-agent System for Data Fusion Techniques Applied to the IoT Enabling Physical Rehabilitation Monitoring. *Applied Sciences* **11**(1), 331 (2021)
14. Shakshuki, E., Reid, M.: Multi-Agent System Applications in Healthcare. *Procedia Computer Science* **52**, 252–261 (2015)
15. Teixeira, E., Fonseca, H., et al.: Wearable Devices for Physical Activity and Healthcare Monitoring in Elderly People. *Geriatrics* **6**(2), 38 (2021)
16. Vargemidis, D., Gerling, K., et al.: Wearable Physical Activity-Tracking Systems for Older Adults. *ACM Trans. Comput. Healthcare* **1**(1) (2020)
17. Vemuri, A., Decker, K., et al.: Multi agent architecture for automated health coaching. *Journal of Medical Systems* **45**(11), 95 (2021)
18. Ziaoddini, K.: Socially Aware Music Recommendation: A Multi-Modal Graph Neural Networks for Collaborative Music Consumption. *arXiv:2511.05497* (2025)
19. Unidade Curricular de Sistemas Multiagente: ASM — Agentes (T1). Apontamentos de aula em PDF, Universidade do Minho (2025/2026)
20. Unidade Curricular de Sistemas Multiagente: ASM — Sistemas Multiagente (T2). Apontamentos de aula em PDF, Universidade do Minho (2025/2026)
21. Unidade Curricular de Sistemas Multiagente: ASM — Normas e Padrões de Comunicação (T3). Apontamentos de aula em PDF, Universidade do Minho (2025/2026)
22. Novais, P., Analide, C.: Agentes Inteligentes. Texto Pedagógico, Grupo de Inteligência Artificial, Departamento de Informática, Universidade do Minho (2006)
23. Wooldridge, M.: Intelligent Agents. In: Weiss, G. (ed.) *Multiagent Systems*, pp. 27–77. MIT Press (1999)
24. Wooldridge, M., Jennings, N.R.: Intelligent Agents: Theory and Practice. *The Knowledge Engineering Review* **10**(2), 115–152 (1995)
25. Weiss, G. (ed.): *Multiagent Systems — A Modern Approach to Distributed Artificial Intelligence*. MIT Press (1999)
26. Durfee, E., Rosenschein, J.: Distributed Problem Solving and Multiagent Systems: Comparisons and Examples. In: Proc. International Workshop on Distributed Artificial Intelligence, Seattle (1994)

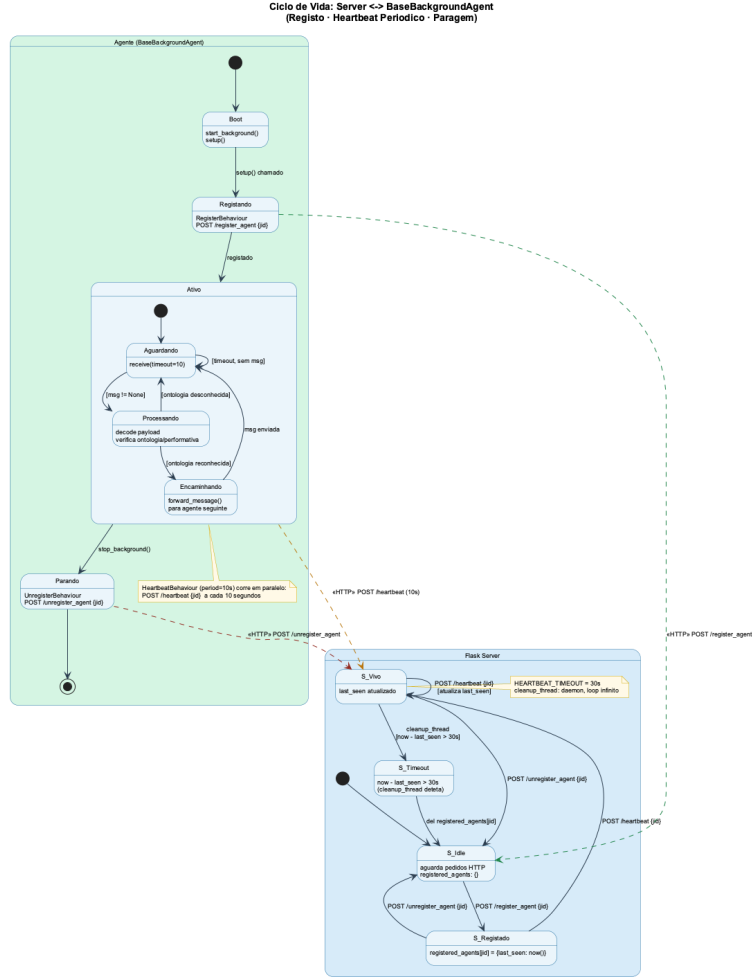


Fig. 11. Statechart — Ciclo de vida Server ↔ BaseBackgroundAgent: registo, heartbeat periódico (period = 10 s) e limpeza por timeout (HEARTBEAT_TIMEOUT = 30 s). As setas a tracejado representam chamadas HTTP fora do canal FIPA-ACL.

Pipeline Inter-Agentes: sensor_activity
FIPA-ACL REQUEST (→) e INFORM (--→) por salto

